GLUTEN FREE LIFE

DOCUMENTACIÓN TÉCNICA

# Manual técnico

Arquitectura, módulos, base de datos, ejecución y pruebas del sistema de gestión y punto de venta.

**Sistema de gestión y punto de venta**
para comercios de productos sin TACC
Para desarrolladores y mantenimiento · Octubre de 2026

# Contenido


1 Objetivo

2 Arquitectura general

3 Tecnologías

4 Estructura del proyecto

5 Módulos principales

6 Flujo técnico de una venta

7 Base de datos

8 Configuración

9 Ejecución

10 Pruebas

11 Datos de demostración

12 Consideraciones de mantenimiento

## 1. Objetivo

Este documento describe la arquitectura, los módulos principales, la lógica de negocio, la base de datos, la ejecución y las pruebas de Gluten Free Life.

## 2. Arquitectura general

El sistema utiliza una arquitectura full-stack dividida en:

```
Navegador
   |
   ├── Frontend React + TypeScript + Vite
   |       ├── Interfaz y navegación
   |       ├── Estado global
   |       ├── IndexedDB para operación offline
   |       └── Cliente HTTP de la API
   |
   └── Backend Node.js + TypeScript + Express
           ├── Autenticación y autorización
           ├── Rutas REST
           ├── Servicios de dominio
           ├── Validación de entrada
           └── PostgreSQL
```

En Docker, el backend sirve también el frontend compilado. El archivo `docker-compose.yml` levanta dos servicios:

- `db`: PostgreSQL 16.
- `app`: imagen que compila y ejecuta frontend y backend.

## 3. Tecnologías

### Frontend

- React 18.
- TypeScript.
- Vite.
- React Router.
- Zustand.
- Tailwind CSS.
- Dexie/IndexedDB para cache y ventas offline.
- `xlsx` para manejo de planillas.
- Recharts para gráficos.

### Backend

- Node.js 22.
- TypeScript.
- Express.
- PostgreSQL.
- `pg` para acceso a datos.
- Zod para validación.
- bcryptjs para contraseñas.
- jsonwebtoken para sesiones.
- ExcelJS y PDFKit para exportaciones.

## 4. Estructura del proyecto

```
frontend/
├── src/App.tsx                Rutas y protección de pantallas
├── src/components/            Componentes reutilizables
├── src/pages/                 Pantallas funcionales
├── src/lib/api.ts             Cliente de la API
├── src/lib/localdb.ts         Persistencia local
├── src/lib/sync.ts            Monitor y sincronización offline
└── src/store/stores.ts        Estado global

backend/
├── src/index.ts               Configuración e inicio de Express
├── src/routes.ts              Endpoints REST
├── src/auth.ts                Autenticación y JWT
├── src/db.ts                  Pool, consultas y transacciones
├── src/http.ts                Errores HTTP
├── src/migrate.ts             Ejecución de migraciones
├── src/seed.ts                Datos de demostración
├── src/services/dominio.ts    Reglas de productos y vencimientos
├── src/services/ventas.ts     Registro y anulación de ventas
├── src/services/caja.ts       Resumen, cierre y reportes de caja
├── src/services/pedidos.ts    Pedidos de compra
├── src/services/ingresos.ts   Ingreso de mercadería
├── src/services/importacion.ts Importación y validación de catálogos
└── src/services/exportar.ts   Exportación de archivos

backend/migrations/
└── 001_init.sql               Esquema inicial de PostgreSQL
```

## 5. Módulos principales

### 5.1 Aplicación frontend

`App.tsx` define las rutas y protege las pantallas internas. Si no existe una sesión, redirige al login.

`Layout.tsx` contiene el menú principal, el indicador de conexión, el usuario activo y el aviso del modo offline.

Las páginas principales son:

- `Dashboard.tsx`: tablero inicial.
- `POS.tsx`: carrito y registro de ventas.
- `Inventario.tsx` y `Productos.tsx`: administración de productos.
- `Vencimientos.tsx`: seguimiento de vencimientos.
- `Compras.tsx`, `PedidoDetalle.tsx` e `IngresoMercaderia.tsx`: compras e ingresos.
- `Caja.tsx`: resumen y cierre diario.
- `Reportes.tsx`: indicadores.
- `Configuracion.tsx`: configuración y estado de conexión.

### 5.2 API y autenticación

`index.ts` configura Express, CORS, JSON, cabeceras de seguridad, rutas y archivos estáticos.

`routes.ts` expone la API bajo el prefijo `/api`. Incluye:

- `GET /api/health`.
- `POST /api/auth/login`.
- Endpoints protegidos para productos, ventas, caja, compras, configuración y reportes.

`auth.ts` compara contraseñas con bcrypt y firma tokens JWT. Las rutas protegidas utilizan el middleware de autenticación.

### 5.3 Acceso a datos

`db.ts` crea el pool de PostgreSQL y contiene helpers para:

- Consultas múltiples ( `q` ).
- Consulta de un único registro ( `q1` ).
- Transacciones ( `tx` ).
- Lectura de configuración.
- Conversión de tipos numéricos y fechas.

Las operaciones que modifican varias tablas se ejecutan dentro de transacciones para mantener la consistencia.

### 5.4 Servicios de dominio

`dominio.ts` concentra reglas compartidas:

- Precio sugerido según costo y margen.
- Estado de stock crítico.
- Reducción y aumento de stock.
- Estados de vencimiento.
- Descuento por vencimiento más próximo.
- Retiro de mercadería vencida.

`ventas.ts` valida y registra ventas. También unifica renglones repetidos, valida cantidades, calcula el total y el vuelto, descuenta stock y permite sincronizar ventas offline.

`caja.ts` calcula totales diarios, separa efectivo y medios digitales, determina diferencias y registra cierres.

## 6. Flujo técnico de una venta

1. El frontend arma el carrito y envía los productos y el medio de pago.
2. El backend valida el cuerpo con `ventaSchema` .
3. Se bloquean los productos involucrados dentro de una transacción.
4. Se unifican líneas repetidas del mismo producto.
5. Se valida que la cantidad sea compatible con `UNIDAD` o `GRANEL` .
6. Se valida el stock disponible.
7. Se calcula el subtotal, total y vuelto.
8. Se inserta la cabecera en `venta` .
9. Se insertan los registros en `detalle_venta` .
10. Se reduce `producto.stock_actual` .
11. Se descuenta el vencimiento vigente más próximo.
12. Se generan alertas si el stock queda crítico.
13. Se confirma la transacción y se devuelve el resultado al frontend.

La sincronización offline utiliza `uuid_cliente` para evitar duplicar una venta reenviada.

## 7. Base de datos

La migración se encuentra en `001_init.sql` .

### 7.1 Tablas principales

| Tabla | Responsabilidad |
| --- | --- |
| `usuario` | Usuarios, credenciales y estado activo |

| | |
|---|---|
| `producto` | Catálogo, precios, stock y tipo de venta |
| `vencimiento_producto` | Cantidades y fechas de vencimiento |
| `venta` | Cabecera de cada operación |
| `detalle_venta` | Productos incluidos en una venta |
| `cierre_caja` | Arqueo y cierre diario |
| `intermediario_logistico` | Proveedor o intermediario |
| `pedido_compra` | Cabecera de pedidos |
| `detalle_pedido` | Productos solicitados en cada pedido |
| `pedido_estado_historial` | Historial de estados |
| `ingreso_mercaderia` | Cabecera de ingresos |
| `detalle_ingreso` | Productos, cantidades y costos recibidos |
| `configuracion` | Parámetros del comercio |
| `alerta_sistema` | Avisos generados por el sistema |

### 7.2 Relaciones

```
usuario 1 ── N venta
usuario 1 ── N cierre_caja
usuario 1 ── N pedido_compra
usuario 1 ── N ingreso_mercaderia

producto 1 ── N vencimiento_producto
producto 1 ── N detalle_venta
producto 1 ── N detalle_pedido
producto 1 ── N detalle_ingreso

venta 1 ── N detalle_venta
pedido_compra 1 ── N detalle_pedido
pedido_compra 1 ── N pedido_estado_historial
pedido_compra 1 ── N ingreso_mercaderia
ingreso_mercaderia 1 ── N detalle_ingreso
intermediario_logistico 1 ── N pedido_compra
```

Las relaciones se implementan con claves foráneas. Los detalles de ventas, pedidos, ingresos e historial utilizan eliminación en cascada cuando corresponde.

### 7.3 Integridad

El modelo utiliza:

- Claves primarias autoincrementales.
- Restricciones `NOT NULL`.
- Valores únicos para login, código de barras y datos técnicos.
- Checks para evitar stocks y cantidades negativas.
- Enumeraciones para tipos de venta, medios de pago y estados.
- Índices para búsquedas por producto, fecha y relación.

## 8. Configuración

En Docker se configura mediante variables de entorno:

| Variable | Uso | Valor habitual |
|---|---|---|
| `POSTGRES_PASSWORD` | Contraseña de PostgreSQL | `gfl_dev` |
| `JWT_SECRET` | Firma de tokens | valor de desarrollo |
| `TZ_NEGOCIO` | Zona horaria comercial | `America/Argentina/Buenos_Aires` |
| `PORT` | Puerto del backend | `4000` |
| `APP_PORT` | Puerto publicado por Docker | `4000` |
| `CORS_ORIGIN` | Origen permitido | `http://localhost:4000` con Docker |
| `STORAGE_DIR` | Archivos generados | `/data/storage` en Docker |

Para desarrollo local, el backend utiliza `backend/.env` y el frontend puede apuntar a otro backend mediante `VITE_API_TARGET`.

## 9. Ejecución

### Docker

Requisitos:

- Docker Engine.
- Docker Compose.
- Git, si el proyecto se obtiene clonando el repositorio.

En Linux, el usuario debe tener permisos para acceder al socket de Docker. La lista de requisitos del entorno también está disponible en `requirements.txt`. Este archivo es únicamente documentación: el proyecto no utiliza Python ni se instala con `pip`.

Desde la raíz:

```
docker-compose up -d --build
```

Con Docker Compose moderno también puede utilizarse:

```
docker compose up -d --build
```

Si Docker devuelve `PermissionError: [Errno 13] Permission denied`, el usuario actual no tiene permisos sobre el servicio Docker. En ese caso, ejecutar:

```
sudo docker-compose up -d --build
```

Con Compose V2:

```
sudo docker compose up -d --build
```

Para configurar el acceso permanente sin `sudo`:

```
sudo usermod -aG docker "$USER"
sudo reboot
```

Después del reinicio se puede comprobar el acceso con:

```
docker ps
```

La aplicación se abre en `http://localhost:4000`.

### Desarrollo local

Base de datos:

```
createdb gfl
```

Backend:

```
cd backend
cp .env.example .env
npm install
npm run migrate
npm run seed
npm run dev
```

Frontend, en otra terminal:

```
cd frontend
npm install
npm run dev
```

El frontend se abre en `http://localhost:5173` y utiliza el proxy `/api` hacia `http://localhost:4000`.

## 10. Pruebas

### Prueba de integración/e2e existente

La prueba se encuentra en `e2e.ts`. Utiliza PostgreSQL real, carga datos de demostración, levanta la aplicación en un puerto dinámico y prueba la API completa.

Para ejecutarla:

```
cd backend
npm test
```

La prueba cubre, entre otros casos:

- Health check y login.
- Rechazo de credenciales incorrectas.
- Protección de endpoints sin token.
- Consulta y filtrado de productos.
- Registro de ventas.
- Cálculo de totales y vuelto.
- Descuento de stock y vencimientos.
- Pedidos y cambios de estado.
- Caja, configuración y reportes.

El resultado esperado contiene líneas de éxito similares a:

```
✓ health y login
✓ catálogo de demostración
✓ R3/R4 venta: descuenta stock y el vencimiento más próximo; vuelto
✓ R3 un renglón repetido se unifica y baja el vencimiento siguiente al agotar el primero
```

El texto exacto puede variar según la cantidad de casos definidos en la versión del archivo.

### Pruebas unitarias

El repositorio actual no contiene una suite separada de pruebas unitarias con Jest, Vitest u otro runner. Las funciones de dominio, como el cálculo de precio y la clasificación de stock, están separadas y son candidatas directas para pruebas unitarias futuras.

Por lo tanto, para una presentación académica debe distinguirse:

- `e2e.ts`: prueba de integración/end-to-end contra API y PostgreSQL.
- Pruebas unitarias aisladas: no incluidas actualmente en el repositorio.

## 11. Datos de demostración

`seed.ts` crea:

- Usuario `admin` con contraseña `1234`.
- Productos de ejemplo.
- Vencimientos.
- Ventas históricas.
- Pedidos e intermediario.
- Configuración inicial.

`npm run seed` borra y reemplaza los datos de negocio. El arranque Docker utiliza `seed --if-empty`, por lo que solamente carga la demostración cuando la base todavía no tiene usuarios.

## 12. Consideraciones de mantenimiento

- No ejecutar `npm run seed` sobre una base productiva.
- Mantener protegidos `JWT_SECRET`, contraseñas y archivos `.env`.
- Hacer respaldos de PostgreSQL y del volumen de almacenamiento.
- Ejecutar migraciones antes de iniciar una nueva versión.
- Validar los builds del frontend y backend antes de desplegar.
- Revisar los logs del servicio `app` ante fallos de inicio.